

DD_ASR-04 - Service Request

Version: 1.0 Status: Draft for review Bundle: 10_other Audience: customer care developers, ticketing developers, workflow developers, work-order developers, frontend developers, QA Related DDs:

- DD_TCK-01 - Ticketing and Case Management
- DD_ASR-01 - Technical Trouble
- DD_ASR-02 - Information Request
- DD_ASR-03 - Complaint
- DD_SUB-WF-UPGRADE-01 - Subscription Upgrade
- DD_SUB-WF-DOWNGRADE-01 - Subscription Downgrade
- DD_SUB-WF-RELOCATION-01 - Subscription Relocation
- DD_FUL-09 - Equipment Swap Fulfillment
- DD_WO-01 - Work Order Module
- DD_OSR-RMA-01 - Equipment Swap and RMA
- DD_EM-CFG-04 - Approval Workflow Catalog
- FOUNDATION_DROOLS - Rules Engine Foundation
- FE-APP-01 - BSS Backoffice Web App
- FE-APP-04 - Customer Self Care Frontoffice App

MVP simplification baseline: ASR-04 is implemented as TCK-01 service-request ticket categories that trigger owning module APIs where needed. It is not a separate request engine, database, or deployment in the first release.

1. Purpose

ASR-04 defines how SOPHIX handles customer service requests.

A service request is a customer-initiated operational action that is not primarily a fault, complaint, or neutral information question. The customer is asking SOPHIX to do something.

Examples:

- reset customer service credentials or SIP profile;
- request package upgrade/downgrade;
- request relocation/address change;
- request equipment swap;
- request contact detail correction;
- request installation appointment change;
- request ownership/contact-person update;
- request cancellation of an open non-critical request;
- request account document/reference re-send;
- request bucket/benefit transfer when the operator supports it.

This DD does not create a new ticketing engine and does not replace owning workflows. Service requests are implemented as configured TCK tickets with ASR-specific request capture, validation, approval, dispatch, and completion tracking. The owning module still executes the business action.

2. Ownership Boundary

Concern	Owner
Ticket lifecycle, comments, assignment, SLA, attachments	TCK-01
Service request category behavior, required fields, approval/dispatch policy	ASR-04 config inside TCK/ASR
Subscription package changes	SUB-WF upgrade/downgrade/migration workflows
Relocation/address move	SUB-WF-RELOCATION-01 and WO shifting flow
Equipment swap execution	FUL-09, OSR-RMA-01, WO support/install flow
Technical repair	ASR-01 / WO-01
Complaint handling	ASR-03
Customer/account master updates	ILM/customer service APIs
Notifications	NOT-01
Configurable approval process, approval tasks, approval callbacks	EM-CFG-04
Configurable request decision rules	Drools rules engine where policy is operator-variable

Rule: ASR-04 owns request intake and tracking. It must call the owning workflow/module to execute the requested action and store the returned external reference. It must not bypass those modules with direct table updates.

Approval rule: ASR-04 does not define approval steps locally. If a service request category requires approval, ASR-04 calls EM-CFG-04 to evaluate and run the configured approval policy. EM-CFG-04 may execute the approval as auto-approval, single-step DB-driven approval, multi-step approval, or Camunda BPMN depending on operator policy.

3. When To Use This Flow

Use ASR-04 when the expected outcome is an operational action.

Scenario	Use
Customer asks a question	ASR-02 Information Request
Customer reports service not working	ASR-01 Technical Trouble
Customer is dissatisfied and wants investigation/response	ASR-03 Complaint
Customer requests package/address/equipment/account/SIP action	ASR-04 Service Request
Customer requests package change already covered by self-service workflow	ASR-04 may intake, but SUB workflow owns execution

4. High Level Flow

```

flowchart TD
    A["Customer requests action"]
    B["Create TCK service request ticket"]
    C["Resolve ASR request type config"]
    D["Validate required fields and eligibility"]
    E{"Approval required?"}
    F["Create approval task / supervisor review"]
    G["Dispatch command to owning module"]
    H["Store external operation reference"]
    I{"Owning operation completed?"}
    J["Wait / sync status"]
    K["Add customer-visible result"]
    L["Resolve TCK ticket"]
    M["Reject / request more info"]

    A --> B --> C --> D
    D -->|Invalid| M
    D -->|Valid| E
    E -->|Yes| F --> G
    E -->|No| G
    G --> H --> I
    I -->|No| J --> I
    I -->|Yes| K --> L
  
```

5. Request Types

Recommended request categories:

Category code	Meaning	Owning module/workflow	Default priority
SIP_PROFILE_RESET	Reset service/SIP profile or credentials when allowed.	Provisioning/FUL/technical service API	P3
PACKAGE_UPGRADE_REQUEST	Customer requests package upgrade.	SUB-WF-UPGRADE-01	P3
PACKAGE_DOWNGRADE_REQUEST	Customer requests package downgrade.	SUB-WF-DOWNGRADE-01	P3
RELOCATION_REQUEST	Customer requests move/address relocation.	SUB-WF-RELOCATION-01 / WO shifting	P2
EQUIPMENT_SWAP_REQUEST	Customer requests CPE/STB/router swap.	FUL-09 / OSR-RMA-01 / WO	P2
CONTACT_UPDATE_REQUEST	Customer requests phone/email/contact-person correction.	ILM/customer service API	P4
APPOINTMENT_CHANGE_REQUEST	Customer asks to reschedule installation/support visit.	WO-01	P3
DOCUMENT_RESEND_REQUEST	Customer asks for invoice/contract/reference re-send.	BIL read/NOT/file storage	P4
BENEFIT_TRANSFER_REQUEST	Customer asks to transfer bucket/benefit if supported.	Billing/wallet/product rules	P3
GENERAL_SERVICE_REQUEST	Other operational request.	Manual triage	P4

6. ASR-Specific Tables

6.1 asr_service_request_type_config

Stores behavior for each service request category.

Use this table to decide required fields, owning module, approval policy, whether self-care may create the request, and whether ASR should wait for an external operation to complete before resolving.

Column	Type	Purpose	Sample
config_id	UUID PK	Config id.	asrtc-001
operator_code	TEXT	Operator/tenant.	WIK
category_code	TEXT	TCK category.	PACKAGE_UPGRADE_REQUEST
default_type_code	TEXT	TCK type.	GENERAL_INQUIRY
owning_module	TEXT	Module/workflow that executes the action.	SUB-WF-UPGRADE-01
command_endpoint	TEXT NULL	Internal API endpoint to call.	/api/subscriptions/{subscription_id}/upgrade-requests
required_fields_json	JSONB	Required request payload fields.	["subscriptionId", "targetPackageRef", "requestedEffectiveDate"]
approval_policy_code	TEXT	NONE, SUPERVISOR, BILLING, TECHNICAL, CUSTOM.	NONE
selfcare_allowed	BOOLEAN	Customer can create from self-care.	true
wait_for_external_completion	BOOLEAN	Ticket waits for owning operation result.	true
customer_confirmation_required	BOOLEAN	Customer must confirm final terms.	true
status	TEXT	ACTIVE, RETIRED.	ACTIVE

Recommended indexes:

- Unique (operator_code, category_code).

6.2 asr_service_request

Stores the structured service request attached to a TCK ticket.

Use this table after a TCK ticket is created and ASR validates the request payload. It is the ASR-side operational record, not the owner of the downstream workflow.

Column	Type	Purpose	Sample
service_request_id	UUID PK	ASR request id.	asr-sr-001
ticket_id	TEXT	TCK ticket id.	tck-20260605-000014
operator_code	TEXT	Operator/tenant.	WIK
category_code	TEXT	Request category.	PACKAGE_UPGRADE_REQUEST
request_status	TEXT	DRAFT, VALIDATED, WAITING_APPROVAL, DISPATCHED, COMPLETED, REJECTED, CANCELLED, FAILED.	DISPATCHED
request_payload_json	JSONB	Validated request fields.	{"subscriptionId":"SUB-700045","targetPackageRef":"pkg-fiber-200m"}
validation_result_json	JSONB	Validation checks/results.	{"eligible":true,"warnings":[]}
created_by_user_id	TEXT	User/system.	usr-care-001
created_at	TIMESTAMP	Creation timestamp.	2026-06-05T09:00:00Z
completed_at	TIMESTAMP NULL	Completion timestamp.	2026-06-05T09:15:00Z

Recommended indexes:

- Unique (ticket_id).
- (operator_code, category_code, request_status).

6.3 asr_service_request_approval

Stores the ASR-04 side reference to a shared EM-CFG-04 approval request.

Use this table when a category requires supervisor, billing, technical, or custom approval before dispatch. The configurable policy, approval steps, approver rules, BPMN process key, user tasks, and final decision are owned by EM-CFG-04. This table lets ASR-04 block dispatch until the shared approval request returns an approved outcome.

Column	Type	Purpose	Sample
approval_id	UUID PK	Approval id.	asra-001
service_request_id	UUID	ASR request id.	asr-sr-001
approval_policy_code	TEXT	EM-CFG-04 policy code used.	SERVICE_REQUEST_TECHNICAL_APPROVAL
approval_request_id	TEXT NULL	EM-CFG-04 runtime approval request id.	apr-sr-001
approval_mode	TEXT	EM-CFG-04 mode: AUTO, SINGLE_STEP, MULTI_STEP, BPMN.	BPMN
camunda_process_instance_id	TEXT NULL	Camunda instance id when EM-CFG-04 uses BPMN.	cam-proc-sr-001
status	TEXT	PENDING, APPROVED, REJECTED, CANCELLED.	APPROVED
assigned_queue_code	TEXT	Approval queue.	CARE_SUPERVISOR
actor_user_id	TEXT NULL	Approver.	usr-supervisor-002
decision_comment	TEXT NULL	Decision note.	Approved after customer confirmed package price.
created_at	TIMESTAMP	Creation timestamp.	2026-06-05T09:02:00Z
decided_at	TIMESTAMP NULL	Decision timestamp.	2026-06-05T09:05:00Z

Recommended indexes:

- (service_request_id, status).
- (approval_request_id).
- (assigned_queue_code, status).

6.4 asr_service_request_dispatch

Stores the call from ASR to the owning module/workflow.

Use this table for idempotency, retry, and traceability when ASR dispatches a service request to SUB, FUL, WO, ILM, billing, or another module.

Column	Type	Purpose	Sample
dispatch_id	UUID PK	Dispatch id.	asrd-001
service_request_id	UUID	ASR request id.	asr-sr-001
owning_module	TEXT	Module/workflow called.	SUB-WF-UPGRADE-01
command_endpoint	TEXT	API endpoint called.	/api/subscriptions/SUB-700045/upgrade-requests
idempotency_key	TEXT	Key used for external call.	asr-sr-001-dispatch-v1
request_payload_json	JSONB	Payload sent to owner.	{"targetPackageRef": "pkg-fiber-200m"}
dispatch_status	TEXT	PENDING, SENT, ACKNOWLEDGED, FAILED, CANCELLED.	ACKNOWLEDGED
owning_ref_id	TEXT NULL	External operation id.	subop-20260605-000019
last_error_code	TEXT NULL	Last failure code.	SUB_PACKAGE_NOT_ELIGIBLE
retry_count	INT	Retry count.	0
created_at	TIMESTAMP	Creation timestamp.	2026-06-05T09:05:00Z
updated_at	TIMESTAMP	Last update timestamp.	2026-06-05T09:06:00Z

Recommended indexes:

- Unique (idempotency_key).
- (owning_module, owning_ref_id).
- (dispatch_status, updated_at).

6.5 asr_service_request_result

Stores the final result returned by the owning module.

Use this table when ASR receives completion/failure from the owning operation and needs to close the customer-facing request.

Column	Type	Purpose	Sample
result_id	UUID PK	Result id.	asrr-001
service_request_id	UUID	ASR request id.	asr-sr-001
owning_module	TEXT	Module that completed.	SUB-WF-UPGRADE-01
owning_ref_id	TEXT	External operation id.	subop-20260605-000019
result_status	TEXT	COMPLETED, REJECTED, FAILED, CANCELLED.	COMPLETED
result_code	TEXT	Owner-specific result code.	UPGRADE_COMPLETED
customer_message	TEXT	Customer-safe final message.	Your package upgrade has been completed.
result_payload_json	JSONB	Structured result details.	{"newPackageRef": "pkg-fiber-200m"}
received_at	TIMESTAMP	Result timestamp.	2026-06-05T09:15:00Z

Recommended indexes:

- Unique (owning_module, owning_ref_id).
- (service_request_id, result_status).

7. APIs

7.1 Create Service Request Ticket

Service requests start with TCK:

POST /api/tickets

```

Idempotency-Key: selfcare-sr-20260605-000014
Content-Type: application/json

{
  "operatorCode": "WIK",
  "typeCode": "GENERAL_INQUIRY",
  "categoryCode": "PACKAGE_UPGRADE_REQUEST",
  "sourceChannel": "SELF_CARE",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "subscriptionId": "SUB-700045",
  "subject": "Package upgrade request",
  "description": "Customer requests upgrade to Fiber 200M."
}

```

7.2 Create Structured ASR Request

```

POST /api/asr/service-requests
Content-Type: application/json

{
  "ticketId": "tck-20260605-000014",
  "operatorCode": "WIK",
  "categoryCode": "PACKAGE_UPGRADE_REQUEST",
  "requestPayload": {
    "subscriptionId": "SUB-700045",
    "targetPackageRef": "pkg-fiber-200m",
    "requestedEffectiveDate": "2026-06-06",
    "customerConfirmedPrice": true
  }
}

```

This API validates config, required fields, self-care permission, and basic eligibility.

7.3 Get Request Guidance

```

GET /api/asr/service-requests/{ticket_id}/guidance

```

Response:

```

{
  "ticketId": "tck-20260605-000014",
  "categoryCode": "PACKAGE_UPGRADE_REQUEST",
  "owningModule": "SUB-WF-UPGRADE-01",
  "requiredFields": ["subscriptionId", "targetPackageRef", "requestedEffectiveDate"],
  "approvalPolicyCode": "NONE",
  "selfcareAllowed": true,
  "waitForExternalCompletion": true,
  "allowedActions": ["VALIDATE", "DISPATCH", "CANCEL"]
}

```

7.4 Validate Request

```

POST /api/asr/service-requests/{service_request_id}/validate

```

Validation calls the owning module's preview/eligibility APIs where available. For package upgrade, call SUB/SIP eligibility. For equipment swap, call OSR/WO eligibility. For appointment change, call WO availability.

7.5 Approve Request

Normal approver action happens through EM-CFG-04 task APIs:

```

POST /api/approvals/tasks/{approval_task_id}/approve
Content-Type: application/json

{
  "decisionComment": "Customer confirmed price and relocation fee."
}

```

When EM-CFG-04 finalizes the approval request, it calls ASR-04:

```

POST /internal/asr/service-requests/{service_request_id}/approval-outcome
Content-Type: application/json

{
  "approvalRequestId": "apr-sr-001",
  "outcome": "APPROVED",
  "decisionComment": "Customer confirmed price and relocation fee."
}

```

ASR-04 then marks `asr_service_request_approval.status = APPROVED` and permits dispatch. If the outcome is REJECTED, ASR-04 marks the service request REJECTED and does not dispatch.

7.6 Dispatch Request

```
POST /api/asr/service-requests/{service_request_id}/dispatch
Idempotency-Key: asr-sr-001-dispatch-v1
```

For package upgrade, ASR calls:

```
POST /api/subscriptions/SUB-700045/upgrade-requests
Idempotency-Key: asr-sr-001-dispatch-v1
Content-Type: application/json

{
  "source": "ASR-04",
  "sourceTicketId": "tck-20260605-000014",
  "targetPackageRef": "pkg-fiber-200m",
  "requestedEffectiveDate": "2026-06-06",
  "customerConfirmedPrice": true
}
```

ASR stores the returned operation id in `asr_service_request_dispatch.owning_ref_id`.

7.7 Receive Or Sync Result

Owning modules may publish result events or expose status APIs. ASR supports both.

```
POST /api/asr/service-requests/results
Content-Type: application/json

{
  "owningModule": "SUB-WF-UPGRADE-01",
  "owningRefId": "subop-20260605-000019",
  "resultStatus": "COMPLETED",
  "resultCode": "UPGRADE_COMPLETED",
  "customerMessage": "Your package upgrade to Fiber 200M has been completed.",
  "resultPayload": {
    "subscriptionId": "SUB-700045",
    "newPackageRef": "pkg-fiber-200m"
  }
}
```

7.8 Resolve Ticket

After successful result, ASR adds a customer-visible TCK comment and resolves the ticket using TCK:

```
POST /api/tickets/{ticket_id}/resolve
Content-Type: application/json

{
  "resolutionCode": "SERVICE_REQUEST_COMPLETED",
  "resolutionSummary": "Package upgrade request completed by SUB-WF-UPGRADE-01."
}
```

8. Events

Event	Topic	When emitted
ServiceRequestCreated	sophix.asr.service-request.created	Structured ASR request created.
ServiceRequestValidated	sophix.asr.service-request.validated	Validation completed.
ServiceRequestApprovalRequired	sophix.asr.service-request.approvalrequired	Approval task created.
ServiceRequestApproved	sophix.asr.service-request.approved	Approval granted.
ServiceRequestDispatched	sophix.asr.service-request.dispatched	Owning module command sent.
ServiceRequestCompleted	sophix.asr.service-request.completed	Owning module completed.
ServiceRequestFailed	sophix.asr.service-request.failed	Owning module failed or rejected.

9. Business Rules

Rule	Description	Reason
R-ASR-SR-01	Service request must use configured TCK category.	Avoid undefined routing.
R-ASR-SR-02	Required fields must be validated before dispatch.	Prevent invalid downstream operations.
R-ASR-SR-03	Self-care can create only categories where <code>selfcare_allowed = true</code> .	Avoid exposing restricted actions.
R-ASR-SR-04	ASR must dispatch through owning module API, not direct DB updates.	Preserve ownership and audit.
R-ASR-SR-05	Dispatch must use idempotency key.	Prevent duplicate package changes, WOs, or swaps.
R-ASR-SR-06	If approval is required, dispatch is blocked until approved.	Governance.
R-ASR-SR-07	Ticket cannot resolve until external operation completes when <code>wait_for_external_completion = true</code> .	Avoid false closure.
R-ASR-SR-08	Failed owning operation must keep the request open or resolve as rejected with customer-safe reason.	Customer clarity.
R-ASR-SR-09	Customer-visible final message is required for self-care requests.	Self-care transparency.
R-ASR-SR-10	Approval policy and tasks must be handled through EM-CFG-04. ASR-04 only stores local approval reference and applies final outcome.	Shared approval governance.
R-ASR-SR-11	Camunda is used only when EM-CFG-04 policy mode is BPMN; simple approvals may be DB-driven by EM-CFG-04 without Camunda.	Operational simplicity.

10. Rule Engine Guidance

Not every ASR-04 rule needs Drools. Fixed structural checks should remain normal application validation:

- required field exists;
- ticket/category exists;
- self-care allowed flag;
- external operation is open/closed.

Use Drools when the decision is operator-variable, combines multiple facts, or changes often without code deployment.

Recommended ASR-04 Drools rule sets:

Rule file	Purpose
<code>asr-service-request-eligibility.drl</code>	Decide whether the request is eligible from category, customer/subscription state, requester role, channel, and request payload.
<code>asr-service-request-approval.drl</code>	Decide whether ASR-04 should call EM-CFG-04 and which <code>approval_policy_code</code> to request.
<code>asr-service-request-dispatch.drl</code>	Decide whether dispatch is allowed now or must wait for approval/customer confirmation.

Example Drools fact:

```
ServiceRequestFact {
    operatorCode = "WIK",
    categoryCode = "EQUIPMENT_SWAP_REQUEST",
    sourceChannel = "BACKOFFICE",
    requesterRole = "CARE_AGENT",
    subscriptionStatus = "ACTIVE",
    swapReasonCode = "CUSTOMER_REQUEST",
    customerConfirmedFee = false
}
```

Example rule:

```
rule "Equipment swap requires technical approval when fee not confirmed"
when
    $r : ServiceRequestFact(
        categoryCode == "EQUIPMENT_SWAP_REQUEST",
        customerConfirmedFee == false
    )
```



```

    $out : ServiceRequestDecision()
  then
    $out.setApprovalRequired(true);
    $out.setApprovalPolicyCode("SERVICE_REQUEST_TECHNICAL_APPROVAL");
  end

```

11. Frontend Usage

11.1 FE-APP-01 Backoffice

Backoffice must support:

- create service request from customer 360, subscription, invoice, WO, and ticket screens;
- show request guidance and required fields from ASR API;
- show eligibility preview from owning module;
- request approval when required;
- dispatch request;
- display external operation status/reference;
- show final customer-safe result;
- resolve through TCK.

11.2 FE-APP-04 Customer Self-Care

Self-care must support only enabled request categories. It should show:

- eligible request types for the logged-in customer/subscription;
- required fields and confirmation checkboxes;
- submitted request status;
- final result or rejection reason.

Self-care must not show internal dispatch errors, stack traces, supervisor notes, or raw workflow variables.

12. Infra Recommendation

ASR-04 should run inside the TCK service/module for a medium budget deployment. No separate ASR service-request pod is required initially.

Component	Recommendation
API	Existing TCK API pods expose /api/asr/service-requests/...
DB	Existing TCK schema/database with ASR service request tables.
Workers	Existing TCK worker pool handles dispatch retry, result sync, SLA scan.
Events	Existing Kafka/MSK topics.
Cache	Optional cache for request type config only.

AWS deployment follows TCK deployment: same ECS/EKS service, RDS schema, MSK/Kafka cluster, CloudWatch logs/metrics, and IAM policies. Add alarms for dispatch failures, stuck approvals, open requests past SLA, and repeated downstream rejection by category.

13. Testing Requirements

13.1 Unit Tests

- Required field validation by category.
- Self-care allowed/blocked category validation.

- Approval requirement blocks dispatch.
- Dispatch idempotency key is stable.
- Closure is blocked while external operation is open.
- Drools decision maps approval-required scenario to EM-CFG-04 policy code where configured.

13.2 Integration Tests

- Create package upgrade request and dispatch to SUB workflow.
- Create relocation request and dispatch to relocation workflow.
- Create equipment swap request and dispatch to FUL/OSR/WO flow.
- Failed downstream operation creates customer-safe failed result.
- Result event resolves ASR request and TCK ticket.
- EM-CFG-04 approval request/callback permits dispatch.

13.3 End To End Tests

- Self-care package upgrade request completes successfully.
- Backoffice equipment swap request waits for WO/OSR completion.
- Appointment change request updates WO schedule and resolves ticket.
- Approval-required request cannot dispatch before approval.

14. Definition Of Done

- Service request categories are seeded in TCK.
- ASR service request config exists per operator/category.
- APIs support guidance, create, validate, approval, dispatch, result sync, and closure validation.
- EM-CFG-04 is used for configurable approval policies; ASR-04 does not hardcode approver steps.
- Drools rule files exist for configurable eligibility/approval/dispatch decisions where operator policy requires them.
- Dispatch uses owning module APIs with idempotency.
- Backoffice can process service requests end to end.
- Self-care can submit only allowed request types.
- Events are emitted for reporting and timeline.
- Tests cover package, relocation, equipment, appointment, and failed dispatch paths.